Maintainer Prioritization in npm Supply Chains: Objectives, Representation Sensitivity, and Metadata Limits

Shahriar Hasan[1*] and Md. Ashikul Islam[2]

[1]Department of ECE, Hajee Mohammad Danesh Science and Technology University, Basherhat, HSTU, Dinajpur, 5200, Bangladesh.
[2]Department of CSE, Hajee Mohammad Danesh Science and Technology University, Basherhat, HSTU, Dinajpur, 5200, Bangladesh.

*Corresponding author(s). E-mail(s): hasanshahriar32@gmail.com;
Contributing authors: ashikulislamcf2000@gmail.com;

Abstract

Maintainer-review programs must choose between reaching distinct repositories, reducing account-origin exposure, and selecting every observed source of an endpoint. We compare these objectives in a retained npm graph of 25,228 identifiers and 12,217 usernames, then test representation choices, heuristic components, and transport to a newly acquired cohort. At budget 100, greedy coverage reaches 5,538 of 7,171 workflow-associated repositories, whereas exact-union ranking reaches 4,407. Their exposure reductions under a uniform single-account-origin model are 11.48% and 33.79%. An exclusion-oriented heuristic selects every observed source for 1,022 repositories; a simpler variant yields 1,024. Across 96 candidate-restricted exact benchmark settings, the original heuristic attains the optimum in 86, with a worst deficit of 66 endpoints. A fresh acquisition from 100 sampled roots supplies 66 workflow-associated repository endpoints: greedy reaches 65 and exact union 42, with exposure shares of 11.19% and 44.95%. Version continuity decreases account–repository pair mass by 3.33% in that cohort, compared with 42.44% in the original graph. A separate automated evidence audit of 120 endpoints acquires 115 pinned repository commits. The results show objective-dependent decisions and population-dependent effect sizes, without establishing compromise probabilities, historical installations, or deployed-control efficacy.

Keywords: Software supply chain security, npm ecosystem, maintainer prioritization, multi-objective optimization, dependency graphs, release authority

1 Introduction

npm dependency and maintainer relations can concentrate downstream reach in a small set of accounts. Existing work establishes maintainer influence, combined reach and the potential value of vetting influential maintainers (Zimmermann et al. 2019; Zahan et al. 2022). A review program must also specify what a limited account budget should accomplish. Reaching many distinct repositories, removing account-origin exposure, and selecting every observed source are different objectives.

Consider two accounts whose portfolios reach many of the same repositories. After selecting the first, the second may add little distinct coverage while remaining a possible origin in a structural exposure model. An endpoint can therefore be covered while retaining many unselected sources. Mathematical definitions establish that these quantities differ; an empirical study must measure how consequential the distinction is and how observation choices affect the comparison.

We compare the objectives on a retained, high-impact-seeded npm graph containing 25,228 identifiers, including bundled contexts, and 12,217 observed usernames. The main endpoint universe contains 7,171 normalized workflow-associated repository keys. These units are neither a registry-wide sample nor observations of installed-code execution. We evaluate established ranking and greedy-coverage baselines (Zimmermann et al. 2019; Nemhauser et al. 1978), exploratory scalarization, and an exclusion-oriented heuristic.

We assess the comparison through component benchmarks, a new acquisition and an automated endpoint-evidence audit. Component ablations and small exact benchmarks test whether the exclusion heuristic's additional scoring is useful. A new acquisition from 100 frozen, sampled roots tests transport to a bounded cohort with fresh dependency, maintainer and workflow observations. A stratified probability sample of 120 endpoints supplies commit-pinned evidence about repository mappings and workflow files. Automated indicators characterize this evidence; their accuracy was not manually validated.

We ask four questions:

- **RQ1:** How large are the observed coverage, additive-exposure and all-source-exclusion tradeoffs at equal budgets?
- **RQ2:** How do representation choices affect evaluation, and what evidence supports the workflow-associated endpoint observations?
- **RQ3:** How sensitive is the exclusion-oriented heuristic to its components, candidate cap and ties, and where does it miss candidate-restricted optima?
- **RQ4:** Which tradeoffs persist in a new bounded acquisition and a common-population comparison?

Our contribution is measured evidence about these decision and observation choices. The additive optimum and standard submodular guarantee are theoretical foundations, not new coverage algorithms. The heuristic is evaluated without a global-optimality claim; its simpler variant slightly improves exclusion on the original graph but underperforms it in the new cohort. The additional acquisition repeats the coverage–exposure tradeoff while substantially changing the size of representation effects. Extended objective sensitivities and historical reconstruction and

metadata-acquisition diagnostics are retained in the appendices rather than presented as operational or forecasting validation.

2 Related Work

Our research intersects five areas of empirical software engineering and software supply-chain security: structural ecosystem measurement, malicious package detection, continuous-integration workflow analysis, registry governance and provenance, and temporal dependency reconstruction.

2.1 Ecosystem Structure and Centrality-Based Prioritization

Prior studies document concentrated dependency reach in open-source package registries. Zimmermann et al. (Zimmermann et al. 2019) analyzed npm's dependency topology, revealing that a tiny core of central packages and maintainers transitively affects a vast majority of the ecosystem. Zahan et al. (Zahan et al. 2022) study metadata weak-link signals, while Liu et al. (Liu et al. 2022) examine dependency-tree vulnerability propagation.

Zimmermann et al. (Zimmermann et al. 2019) already studied maintainer-union reach, greedy coverage, and vetting. Wang (Wang 2026) evaluates package-node protection in cascade simulations. Our contribution is an empirical comparison of coverage, account-origin exposure, and all-source exclusion at workflow-associated repository endpoints. We evaluate established baselines and a stated heuristic, without claiming a new coverage algorithm or demonstrating that PageRank fails in an experiment where it was not evaluated.

2.2 Malicious Package Benchmarks and Detection Limits

A parallel line of security research focuses on detecting malicious packages published via typosquatting, dependency confusion, and account takeovers (Ohm et al. 2020; Sejfia and Schäfer 2022). Benchmark evaluations, such as Guo et al.'s extensive study of npm malware detectors (Guo et al. 2026), show that static analysis tools have limitations due to obfuscation, dynamic runtime execution, and environment-keyed payloads.

Because reactive post-publication malware detection suffers from non-trivial evasion rates, proactive hardening of legitimate, high-leverage maintainer accounts is a complementary research direction (Schorlemmer et al. 2024; Kalu et al. 2025). Our comparison concerns a hypothetical account-review budget; it does not evaluate a deployed prevention program.

2.3 Continuous-Integration Workflows and Reachability Limits

Software supply-chain risks manifest concretely in downstream CI/CD pipelines. Koishybayev et al. (Koishybayev et al. 2022) and Muralee et al. (Muralee et al. 2023) demonstrated that GitHub Actions workflows frequently execute third-party build scripts with access to repository secrets, cloud deployment credentials, and package

publish tokens. Incident reports motivate inspecting actual artifacts and authorization evidence; they do not validate the workflow-associated endpoints in our graph.

Recent empirical studies have emphasized that package-level dependency relationships do not equal actual code execution. Weeraddana, Alfadel, and McIntosh (Weeraddana et al. 2024) studied CI time spent updating unused npm dependencies. Shen et al. (Shen et al. 2025) reported that 99% of client vulnerabilities in their studied corpus arose from dependencies and 81.26% of package-level findings were false positives under finer analysis. These corpus-specific results do not establish registry-wide false-positive rates. Jayasuriya et al. (Jayasuriya et al. 2025) observed similar granularity effects when analyzing syntactic breaking changes across versions. We measure sensitivity to package-name and exact-version edge unions, while explicitly retaining the gap between topological reach and installed-code evidence.

2.4 Registry Governance, Cryptographic Provenance, and Signing

In response to account compromises, registries and standards bodies have introduced cryptographic signing and provenance attestations. Schorlemmer et al. (Schorlemmer et al. 2024) audited signing across Maven, PyPI, Docker Hub, and Hugging Face, finding that adoption remains minimal and unevenly distributed. Kalu et al. (Kalu et al. 2025) interviewed industry practitioners, revealing significant operational friction in managing cryptographic keys and configuring automated pipeline attestations. Bloom Security's SlugFest disclosure concerns mutable repository-name bindings before a first trusted publish, rather than a break of OIDC cryptography (Bloom Security 2026). GitHub announced expiry of unvalidated bindings on October 2 (GitHub 2026); reported exposure is not evidence that all affected packages were maliciously published. Pillar's October 19 OpenCode report describes a September 28 dependency fix without a CVE (Cohen 2026), illustrating a coverage gap for CVE-indexed detection.

Simultaneously, He, Vasilescu, and Kästner (He et al. 2025) demonstrated that local dependency version pinning is insufficient to protect clients against upstream attacks, as adversaries exploit unpinned sub-dependencies and registry-level re-resolutions. These findings motivate examining account controls alongside version and artifact evidence; they do not establish control efficacy in our graph.

2.5 Temporal Reconstruction and Empirical Standards

A critical methodological vulnerability in supply-chain studies is future-leakage bias—evaluating historical attacks using contemporary registry graphs. Pinckney et al. (Pinckney et al. 2023) introduced `npm-follower` to mirror registry event streams, while Rahman et al. (Rahman et al. 2026) released `CHRONO-RESOLUTION` to reconstruct exact dependency trees at release points. Shafin and Ahmed (Shafin and Ahmed 2026) highlighted the necessity of time-indexed forecasting.

Our reporting is informed by the ACM SIGSOFT Empirical Standards (Ralph et al. 2020). We report fixed-graph budget comparisons and repeated random orders as descriptive analyses, rather than treating them as independent ecosystem samples.

Table 1 compares the closest empirical studies and their observation units.

Table 1: Closest empirical work and the additional question addressed here. Different observation units prevent direct comparison of reported security percentages

Study	Population	Main observations	Validation basis	Additional question here
(Zimmermann et al. 2019)	npm dependency and maintainer ecosystem	Maintainer influence, combined reach and vetting simulations	Ecosystem-scale longitudinal dependency analysis	Joint C/H/P costs, endpoint definitions and representation-sensitive selection
(Zahan et al. 2022)	1.63 million npm packages	Metadata weak-link signals	Metadata analyses and survey of 470 developers	Explicit structural objectives rather than fitted compromise signals
(Weeraddana et al. 2024)	1,487 projects and 20,743 dependency-update commits	Unused dependencies and CI resource waste	Dependency-use analysis and measured CI builds	Evidence limits of workflow-presence proxies; this study does not measure job execution
This study	Retained seeded graph; new 100-root cohort; 120-endpoint audit	C/H/P tradeoffs, heuristic ablations and representation effects	Graph consistency checks, restricted exact benchmarks and fresh acquisition	Transport of decision tradeoffs and explicit limitations of observations

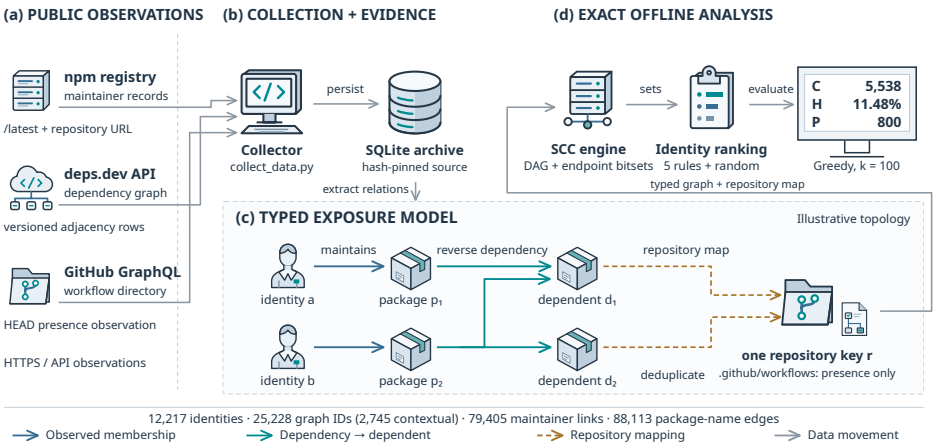


Fig. 1 Measurement architecture: (a) public observations, (b) collection and retained evidence, (c) typed exposure model, and (d) offline analysis. Icons denote logical roles and topology is illustrative. Counts and displayed greedy outcomes are empirical; workflow presence does not establish execution

3 Data Corpus and Graph Construction

3.1 Collection and Units

The collector seeds a high-impact npm list, expands dependencies through deps.dev (Open Source Insights 2026), requests registry metadata, and checks linked GitHub repositories for a `.github/workflows` directory at HEAD. Queue dates span June 13–16, 2026, but individual fields lack a common acquisition cutoff. The retained archive analyzed on October 20 contains 25,228 graph identifiers, 22,340 version-metadata rows, 12,217 maintainer usernames, and 79,405 package–maintainer relations. Maintainer membership is not a historical publisher log.

Among these identifiers, 2,745 encode bundled contexts rather than independently registered packages. The 121,885 retained version-edge rows project to 88,113 distinct name adjacencies. Stored `DIRECT`/`INDIRECT` labels describe a target node relative to an extraction root; their 48,705/73,180 row counts do not partition global direct and transitive edges (Open Source Insights 2026). We reverse every explicit declaring-to-dependency adjacency to measure downstream reach.

Figure 1 summarizes the collection and analysis architecture.

The 15,233 workflow-positive package names map to 7,171 normalized repository keys. Owner/name pairs are case-folded and trailing transport `.git` is removed. This is lexical deduplication, without redirect or commit verification. There are 3,081 identifiers without maintainer mappings and 3,772 without completed workflow flags. Zero flags conflate directory absence with repository unavailability. Workflow endpoints require a positive observation; missing and zero flags are not evidence of safety. Coverage uses all 7,171 repository endpoints. Only 7,158 have an observed source identity; the other 13 are excluded from source-exclusion counts.

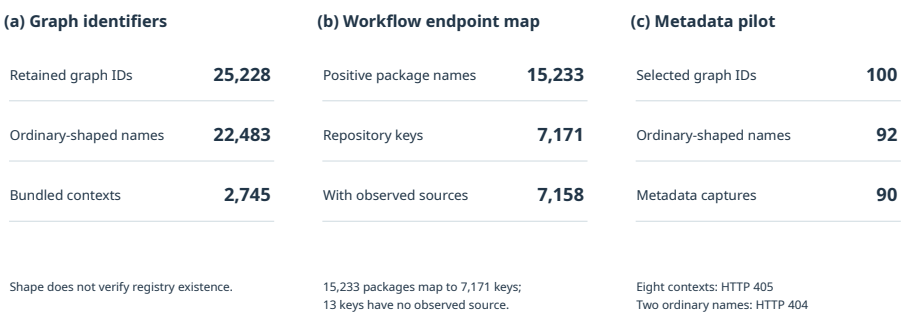


Fig. 2 Observation units and metadata acquisition. Panels describe separate populations; their missingness counts must not be added across panels. Context identifiers are retained graph units, not separate registry packages. The metadata pilot sampled 100 identifiers before requests; its observed request outcomes are shown separately

Figure 2 separates graph units, observed workflow endpoints, and the metadata-pilot sampling frame. This distinction matters because directory observation and metadata acquisition do not validate a historical installation or a future outcome.

3.2 Name Projection and Exact-Version Continuity

The name graph merges retained versions sharing an identifier. The sensitivity graph retains 40,806 exact (package, version) nodes and joins paths only at matching version keys. A maintained package seeds every retained version; without a version node, it contributes only its eligible seed endpoint. Both representations use the same package-level workflow flags. Neither reproduces npm installation or historical lockfile behavior: contexts and acquisition times remain mixed.

3.3 Automated Endpoint Evidence Audit

Before new audit requests, we freeze 120 endpoints drawn without replacement from the 7,158 repositories with observed sources. Four source-count strata contain 1,716 endpoints with one source, 1,473 with two to five, 1,959 with six to fifty, and 2,010 with more than fifty; each contributes 30 sampled repositories. Inclusion probabilities and design weights are retained. Failed observations are not replaced. Equal stratum allocation means the unweighted pooled support percentage is not an estimate for the full frame.

For each endpoint, acquisition requests its original package/version registry metadata, resolves a repository commit, and retrieves workflow files, candidate package manifests and root lockfiles at that commit. It records response states, timestamps, commit IDs and file hashes. Up to twenty workflow files and twelve manifest/lockfile candidates per repository are retained under the frozen request limits. Mapping support requires matching evidence for every package/version observation assigned to the endpoint; monorepo paths and unresolved reusable actions can leave labels unknown.

Table 2: Automated evidence indicators in the stratified 120-endpoint sample

Evidence criterion	Candidate support	Unresolved
All package mappings matched	106	14
Workflow file acquired	115	5
Aligned install directive	88	32
Relevant root lockfile	70	50
Dependency/version example	17	103

The 115 acquired commits are current-time observations. Unresolved states are not negative or safe outcomes; automated indicators were not manually validated. All mappings assigned to an endpoint must match for the first criterion.

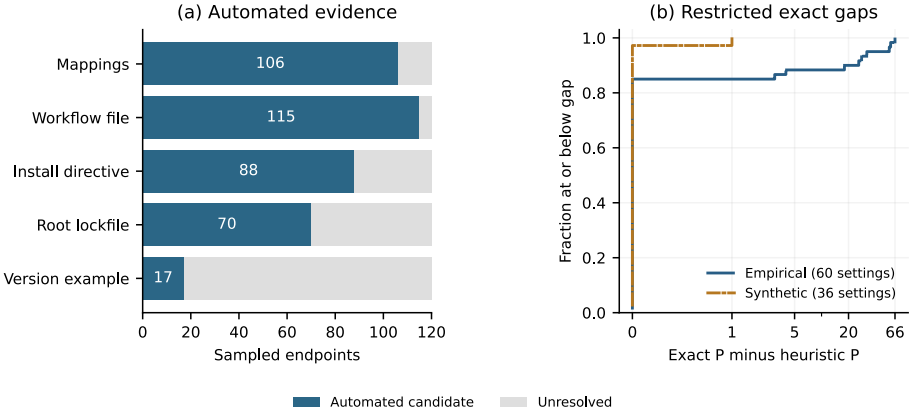


Fig. 3 Evidence diagnostics: (a) automatically extracted endpoint indicators and unresolved cases; (b) empirical and synthetic candidate-restricted exact exclusion gaps. Panel populations and inferential meanings differ. The exact settings reuse instances across budgets

The automated pass records 172 version-specific registry-request outcomes and 115 pinned repository commits. It flags 106 endpoints as mapping-support candidates, 115 with workflow files, 88 with aligned installation directives, 70 with root lockfiles, and 17 with dependency/version examples. Table 2 and Figure 3(a) describe these machine-generated evidence candidates. These indicators were not manually validated and are not estimates of historical mapping precision.

Current-commit evidence cannot retrospectively validate the original undated HEAD observations. Static installation commands do not establish executed jobs, and a lockfile example does not validate every transitive path. Frozen original account selections are evaluated on progressively stricter machine-evidence subsets as provisional diagnostics in the replication materials. Full source sets are preserved. Design-weighted C/P totals and H pair-mass ratios are distinguished from conditional subset results; unresolved-included/excluded H scenarios are not claimed as sharp bounds.

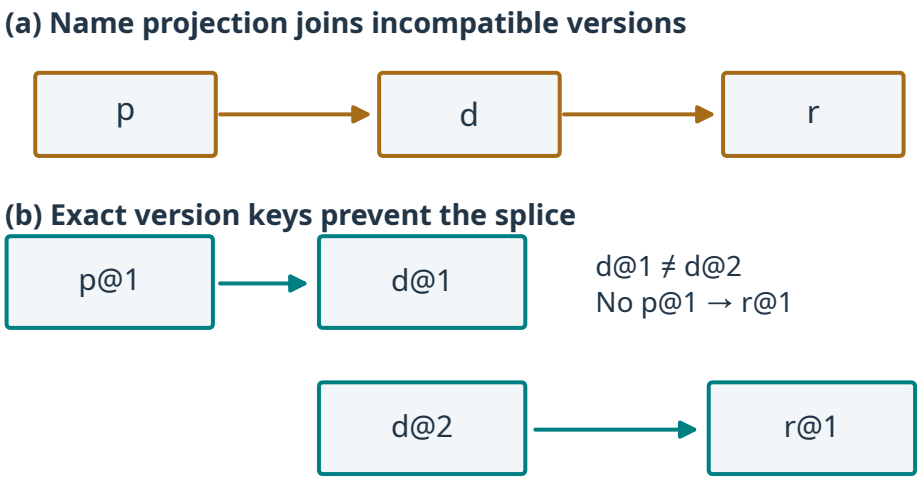


Fig. 4 Illustrative version splicing. Collapsing intermediate versions creates a name-level path without an exact-version counterpart. Reverse-reach arrows do not describe a measured installation

Figure 4 illustrates how version merging can introduce a path absent from the exact-version graph.

3.4 Cycles, Mapping Disagreements and Contexts

The name graph has 82 nontrivial strongly connected components (SCCs) containing 403 nodes; the exact-version union has 68 containing 304 nodes. The largest component has 61 nodes in each. SCC condensation preserves reach and permits exact bitset propagation over a directed acyclic graph; the implementation uses arbitrary-precision Python integers, not fixed 64-bit sets. This is a computation technique, not an intervention that removes cycles.

The URL audit finds 658 identifier-level parser disagreements. Excluding 60 inconsistent positive observations reduces the workflow repository universe to 7,147; greedy at budget 100 gives $C = 5{,}524$ and $P = 799$. We discard ambiguous flags rather than transferring them to a corrected URL without reacquisition. Removing bundled-context nodes preserves all 462,170 primary account–repository incidences for fixed selections, although workflow-package incidence changes. That invariance does not apply automatically to every endpoint definition or individual-package ranking.

Figures were generated programmatically with Matplotlib and editable vector artwork; numerical plot data are included in the replication artifact.

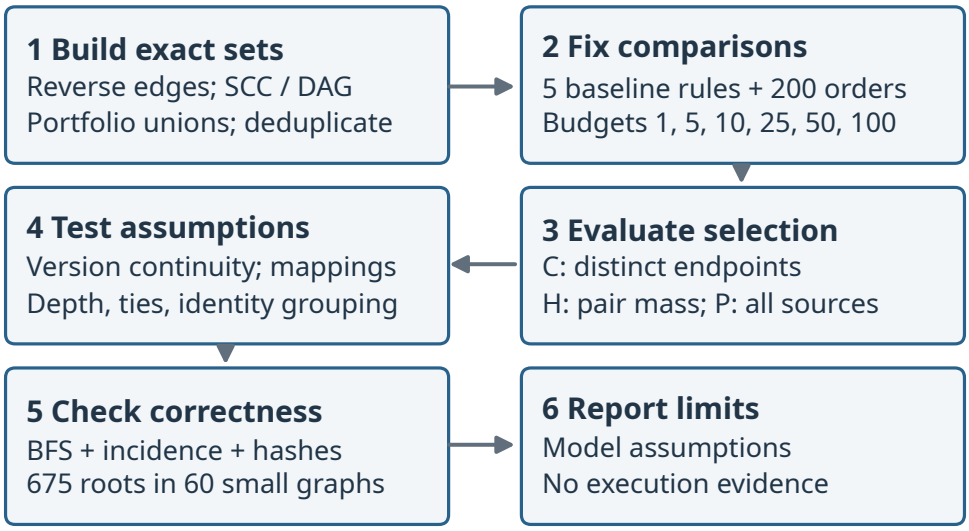


Fig. 5 Analysis methodology and correctness checks. All evaluation quantities share the same observed graph; additional sensitivities do not constitute independent outcome samples

4 Objectives and Optimization Framework

Maintainer prioritization in package registries represents a multi-objective decision problem under a fixed hardening budget $k = |S|$. Let M denote the universe of maintainers ($|M| = 12,217$) and R_{wf} denote the universe of downstream workflow repositories ($|R_{\text{wf}}| = 7,171$). For each maintainer $m \in M$, let $E_m \subseteq R_{\text{wf}}$ denote the set of repositories reachable from packages associated with m, and let $\text{Sources}(r) = \{m \in M : r \in E_m\}$ denote the upstream maintainer set of repository r.

Figure 5 summarizes the analysis and consistency checks.

4.1 Formal Objective Definitions

We formalize three fundamental defensive quantities:

4.1.1 Submodular Distinct Coverage ($C(S)$)

Defenders seeking broad ecosystem review prioritize distinct downstream client repositories:

$$C(S) = \left| \bigcup_{m \in S} E_m \right| \tag{1}$$

Property: $C(S)$ is normalized, monotone non-decreasing, and submodular. For any subsets $A \subseteq B \subseteq M$ and element $m \notin B$:

$$C(A \cup \{m\}) - C(A) \geq C(B \cup \{m\}) - C(B) \tag{2}$$

The classic result of Nemhauser, Wolsey, and Fisher (Nemhauser et al. 1978) establishes that the greedy algorithm selecting $m^* = \arg\max_{m \notin S} |E_m \setminus \bigcup_{u \in S} E_u|$ achieves a $(1 - 1/e) \approx 63.2\%$ approximation of the optimal coverage.

4.1.2 Modular Scenario Exposure Reduction ($H(S)$)

Suppose exactly one account is the hypothetical origin and has a fixed nonnegative weight w_m. A perfect control removes that origin when selected while leaving all other paths unchanged. We measure its share of weighted exposure as:

$$H(S) = \frac{\sum_{m \in S} w_m |E_m|}{\sum_{v \in M} w_v |E_v|} \tag{3}$$

Property: $H(S)$ is normalized, monotone, and strictly *modular* (additive linear). For any disjoint sets $A, B \subseteq M$, $H(A \cup B) = H(A) + H(B)$. Consequently, exact-union ranking (sorting maintainers in descending order of weighted reach $w_m|E_m|$) is mathematically guaranteed to achieve the exact global optimum for $H(S)$ at any budget k.

4.1.3 Supermodular All-Source Exclusion ($P(S)$)

Under the same idealized model, an endpoint has no remaining observed source when every member of its nonempty source set is selected:

$$P(S) = \sum_{r \in R_{\text{wf}}} \mathbb{I}[\varnothing \neq \text{Sources}(r) \subseteq S] \tag{4}$$

Property: $P(S)$ is *supermodular*. Selecting maintainer m contributes zero gain to r until its final unselected source is selected. For $A \subseteq B$ and $m \notin B$,

$$P(A \cup \{m\}) - P(A) \leq P(B \cup \{m\}) - P(B). \tag{5}$$

4.1.4 Strict Zero-Anchored Threshold Exclusion (P_k)

Let $R_+ = \{r \in R_{\text{wf}} : \text{Sources}(r) \neq \varnothing\}$, $n_r = |\text{Sources}(r)|$, and $t_r(S) = |\text{Sources}(r) \setminus S|$. We define the hypothetical strict threshold count

$$P_k(S) = \sum_{r \in R_+} \mathbb{I}\left[t_r(S) = 0 \ \vee \ (n_r \geq k \ \wedge \ t_r(S) < k)\right]. \tag{6}$$

For $n_r < k$, only complete source exclusion is credited. For $n_r \geq k$, fewer than k remaining sources suffice under the hypothetical threshold rule. Thus $P_k(\varnothing) = 0$,

$P_1(S) = P(S)$, and every counted endpoint has a hardened source. The function is monotone in S for fixed k, but need not be monotone in k: an endpoint with $n_r = 2$ and $t_r(S) = 1$ is counted at $k = 2$ and not at $k = 3$. The superficially compact condition $t_r(S) < \min(k, n_r)$ is not equivalent to Eq. 6. No claim about the submodularity or supermodularity of P_1 is extended to general P_k.

The cardinality guard removes zero-budget credit but does not supply missing authorization semantics. It is an explicit modeling convention, not evidence that a repository actually implements a k-person approval policy; upstream source sets may span independently compromised packages.

4.2 Proposition 1: Monotone Submodularity of Hybrid Scalarization $J_\alpha(S)$

To examine choices between coverage and conditional exposure, we evaluate the standard scalarization:

$$J_\alpha(S) = \alpha \cdot \frac{C(S)}{|R_{\text{wf}}|} + (1 - \alpha) \cdot H(S) \tag{7}$$

where $\alpha \in [0, 1]$ parameterizes defensive posture between broad coverage ($\alpha = 1$) and scenario exposure reduction ($\alpha = 0$).

Proposition 1. *For any $\alpha \in [0, 1]$, the objective $J_\alpha(S)$ is monotone non-decreasing and submodular.*

Proof Let $A \subseteq B \subseteq M$ and $m \notin B$. The marginal gain is:

$$\Delta J_\alpha(m \mid S) = \alpha \frac{|E_m \setminus \bigcup_{u \in S} E_u|}{|R_{\text{wf}}|} + (1 - \alpha) \frac{w_m |E_m|}{\sum_{v \in M} w_v |E_v|} \tag{8}$$

The second term is constant with respect to S (modular), so its marginal difference across A and B is exactly zero. The first term satisfies submodularity by set-theoretic inclusion:

$$|E_m \setminus \bigcup_{u \in A} E_u| \geq |E_m \setminus \bigcup_{u \in B} E_u| \tag{9}$$

Because the sum of a submodular set function and a modular set function is submodular, $J_\alpha(S)$ is submodular for all $\alpha \in [0, 1]$. By the Nemhauser bound (Nemhauser et al. 1978), accelerated lazy greedy optimization achieves at least a $(1 - 1/e)$ approximation of the optimum scalarized objective. □

4.3 Supermodular Minimum-Remaining-Source Heuristic for $P(S)$

Because $P(S)$ is supermodular, greedy coverage and exact union achieve all-source exclusion purely as an unoptimized side-effect. To directly optimize $P(S)$, we design an adaptive minimum-remaining-source heuristic. At each step, let $\text{rem}(r) = \text{Sources}(r) \setminus S$. Candidate maintainers are drawn from endpoints with one to ten remaining sources;

if none exist, all unselected maintainers are considered. The implementation maximizes a lexicographic tuple: (i) the number of endpoints immediately completed, (ii) the sum of $1/|\mathrm{rem}(r)|$ over affected endpoints with more than one remaining source, (iii) marginal coverage, and (iv) ascending maintainer index. This is a heuristic with no global optimality guarantee for $P(S)$.

4.4 Heterogeneous Maintainer Risk Formulations

We evaluate three structural models of non-uniform maintainer risk:

1. **Single-Maintainer Bottlenecks ($w_m^{\mathbf{sole}}$):** Packages with $|M(p)| = 1$ have one observed maintainer; actual review practices are unobserved. We weight maintainers by $w_m^{\mathrm{sole}} = 1 + |\{p \in P_m : |M(p)| = 1\}|$.
2. **Log-Portfolio Activity ($w_m^{\mathbf{log}}$):** As an assumed portfolio-size sensitivity, define: $w_m^{\log} = 1 + \log_2(1 + |P_m|)$.
3. **Package-Level Sole-Maintainer Reach ($H_{\mathbf{pkg\text{-}sole}}$):** Instead of weighting all reachable repositories of maintainer m, we isolate repositories reachable strictly through m's sole-maintained packages: $E_m^{\mathrm{sole}} = \bigcup_{p \in P_m, |M(p)|=1} \mathcal{D}(p)$.

4.5 Analytic Expectations and Empirical Standard Errors

Under a finite population of $|M| = 12,217$ maintainers, random selection of budget k yields exact expected coverage and exposure under the hypergeometric distribution:

$$E[H_{\mathrm{rand}}(k)] = \frac{k}{|M|}, \tag{10}$$

$$E[C_{\mathrm{rand}}(k)] = \sum_{r \in R_{\mathrm{wf}}} \left(1 - \frac{\binom{|M| - |\mathrm{Sources}(r)|}{k}}{\binom{|M|}{k}}\right). \tag{11}$$

For $k = 100$, exact finite-population expectations are $E[C] = 1,716.8$ and $E[H] = 0.819\%$. Across 200 simulation seeds, the sample mean coverage is 1,680.6 (sample standard deviation $\mathrm{SD}_C = 530.7$, Monte Carlo Standard Error $\mathrm{MCSE}_C = 37.5$), and sample mean exposure reduction is 0.80% ($\mathrm{SD}_H = 0.373$ pp, $\mathrm{MCSE}_H = 0.026$ pp). Replicated offline execution across 3 independent runs averaged 39.50 s wall-clock time with 1,038.43 MiB peak RSS.

4.6 Use of AI Assistance

OpenAI Codex was used to assist with manuscript drafting and revision, analysis-code development and checking, and literature-source checks. The authors are responsible for the manuscript and its interpretation, code, and references.

5 Core Tradeoffs and Representation: RQ1 and RQ2

We first report our empirical findings for Research Questions 1 and 2 on the retained 25,228-identifier graph, spanning multi-budget trade-offs, multi-homing mechanics, and representation fidelity.

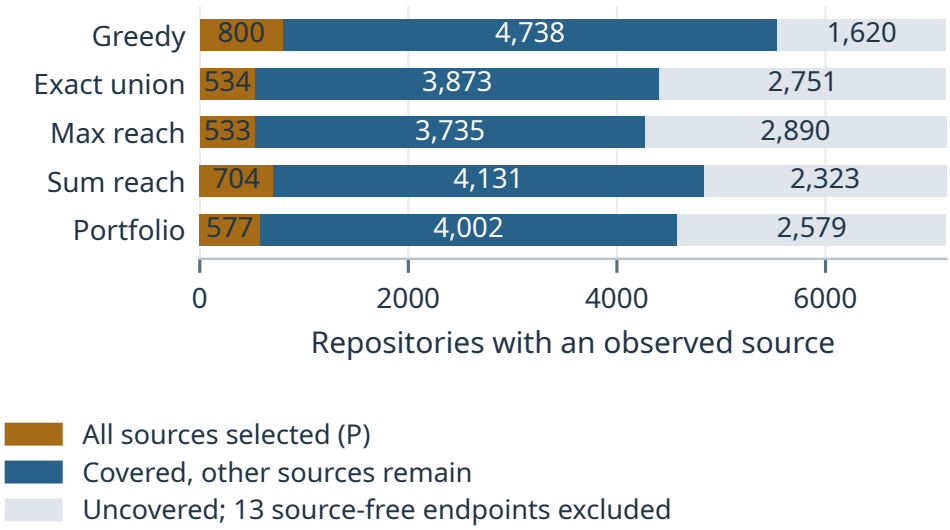


Fig. 6 Budget-100 repository partition. Each bar separates all-source exclusion, coverage with residual sources, and uncovered endpoints. The 13 source-free endpoints are omitted. Structural coverage and complete exclusion differ for every baseline

Figure 6 partitions covered endpoints by whether any unselected source remains.

5.1 RQ1: Coverage vs. Additive Exposure Trade-Off

Figure 7 shows the coverage and exposure trajectories. Table 3 reports empirical prioritization outcomes across maintainer budgets $|S| \in \{1, 5, 10, 25, 50, 100\}$. The top 100 maintainers represent only 0.819% of the 12,217 unique maintainers in our ecosystem, yet their prioritization commands vast reach over downstream workflow repositories.

At budget $|S| = 100$, **Greedy Coverage** achieves $C(S) = 5{,}538$ repositories (77.23%), establishing a descriptive lead of +1,131 repositories (+15.77 percentage points) over **Exact Union** ($C(S) = 4{,}407$, 61.46%). However, when evaluated under scenario-weighted additive exposure reduction $H(S)$, Exact Union achieves 33.79%, outpacing Greedy Coverage (11.48%) by +22.30 **percentage points** (2.94× relative advantage).

Exact union's H advantage follows from its additive objective under equal costs. The experiment measures the accompanying loss in distinct coverage. Greedy offers the largest measured coverage among the compared baseline selections; it is an approximation to maximum coverage, not a proved global optimum. Portfolio size gives 63.85%

Table 3: Workflow-repository outcomes from the retained baseline experiment. Random entries are 200-order sample means and Monte Carlo standard errors, not population confidence intervals

Budget	Metric	Portfolio	Sum	Max	Exact union	Greedy	Random mean ± MCSE
1	C (%)	23.43	33.02	28.75	35.46	35.46	0.65 ± 0.19
1	H (%)	0.36	0.51	0.45	0.55	0.55	0.01 ± 0.00
1	P	1	26	2	269	269	0.28 ± 0.15
5	C (%)	46.00	49.25	42.14	50.96	54.00	2.61 ± 0.36
5	H (%)	1.16	2.15	2.23	2.45	2.09	0.04 ± 0.01
5	P	316	325	80	329	327	0.69 ± 0.17
10	C (%)	47.90	51.75	42.81	54.53	60.40	4.73 ± 0.46
10	H (%)	1.59	2.68	4.06	4.48	3.79	0.08 ± 0.01
10	P	391	399	111	343	351	1.21 ± 0.19
25	C (%)	56.71	60.95	45.29	59.74	67.69	10.15 ± 0.55
25	H (%)	4.38	7.45	9.00	9.86	6.08	0.21 ± 0.01
25	P	430	480	116	427	514	2.86 ± 0.26
50	C (%)	62.00	63.78	45.56	60.26	72.46	16.73 ± 0.56
50	H (%)	6.14	13.72	17.19	18.09	8.73	0.43 ± 0.02
50	P	503	576	117	441	638	5.97 ± 0.38
100	C (%)	63.85	67.42	59.52	61.46	77.23	23.44 ± 0.52
100	H (%)	7.96	21.98	33.59	33.79	11.48	0.80 ± 0.03
100	P	577	704	533	534	800	12.78 ± 1.42

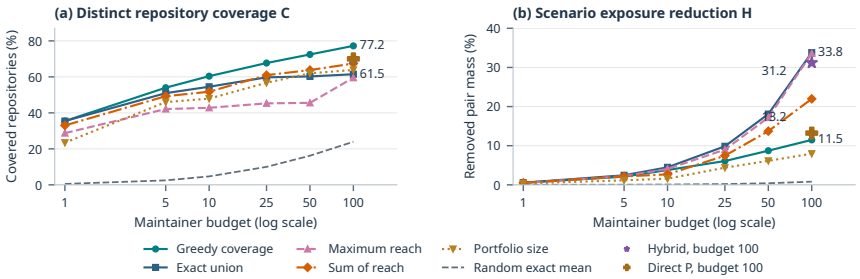


Fig. 7 Multi-budget trade-off trajectories across maintainer budgets $|S| \in [1, 100]$: (a) submodular coverage $C(S)$, (b) modular scenario exposure reduction $H(S)$, using five deterministic rules and the exact random expectation. Budget-100 markers show hybrid and direct-exclusion selections

coverage and 7.96% H at this budget, showing that portfolio count and measured downstream reach need not agree.

5.2 Residual Sources and Single-Source Fragility

A repository can have many source accounts even when a single account already covers it. Across the 7,158 repositories with observed sources, mean source cardinality is 64.57 and the maximum is 1,502. These are graph source identities, not measured permissions or independent compromises. Figure 8 shows their distribution and the remaining-source distributions after budget-100 selections.

Greedy's $P = 800$ is 11.18% of source-reachable repositories; exact union gives $P = 534$. Consequently, 6,358 source-reachable repositories retain at least one unselected

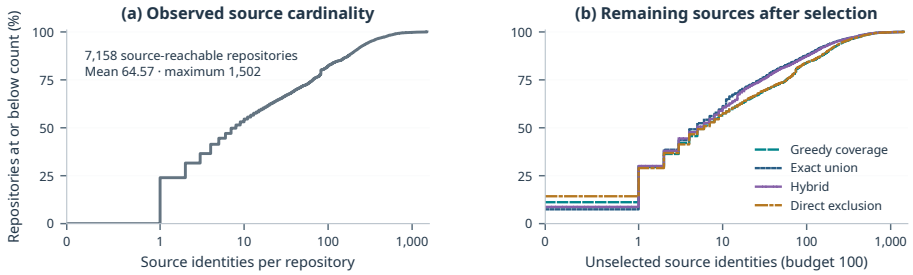


Fig. 8 Observed source multiplicity and residual-source distributions at budget 100. All curves use the same 7,158 source-reachable repositories; 13 source-free endpoints are excluded. Zero remaining sources identify all-source exclusions. Horizontal scales are logarithmic above one. Account origins are structural scenarios, not observed compromises

source after greedy selection. A four-source illustration would retain three source identities when one is selected, but that fraction is not an estimated compromise probability.

5.3 RQ2: Version-Continuity Representation and Selection Penalties

Figure 9 summarizes pair mass and budget-100 outcomes under the two representations. Comparing the package-projected name graph against the observed exact-version edge union reveals substantial sensitivity to representation:

1. **Exposure Shrinkage:** Preserving version continuity contracts total maintainer-repository exposure pairs from 462,170 to 266,016—an exact reduction of 42.44%.
2. **Root Scope Changes:** 8,587 out of 25,228 graph identifiers (34.04%) exhibit divergent downstream reach scopes between name-collapsed and version-continuous representations.
3. **Cycle Condensation:** Kosaraju condensation detects 403 nodes in 82 nontrivial strongly connected components in the name graph, compared to 68 cyclic components across 40,806 version nodes in the version-continuous graph.
4. **Version-Splicing Witnesses:** Retained representation diagnostics include five illustrative roots whose name-level paths lack a continuous counterpart in the retained version graph: `pseudomap`, `is`, `foreach`, `indexof`, and `has-color`.

5.3.1 The Cross-Representation Selection Penalty

When maintainers are selected based on name-projected heuristics but evaluated on the observed version-continuous graph, measured exposure changes. For the `sum_package_reach` heuristic at budget 100, ranking directly on version-continuous topologies captures 34.34% of version exposure, compared to only 20.91% when using

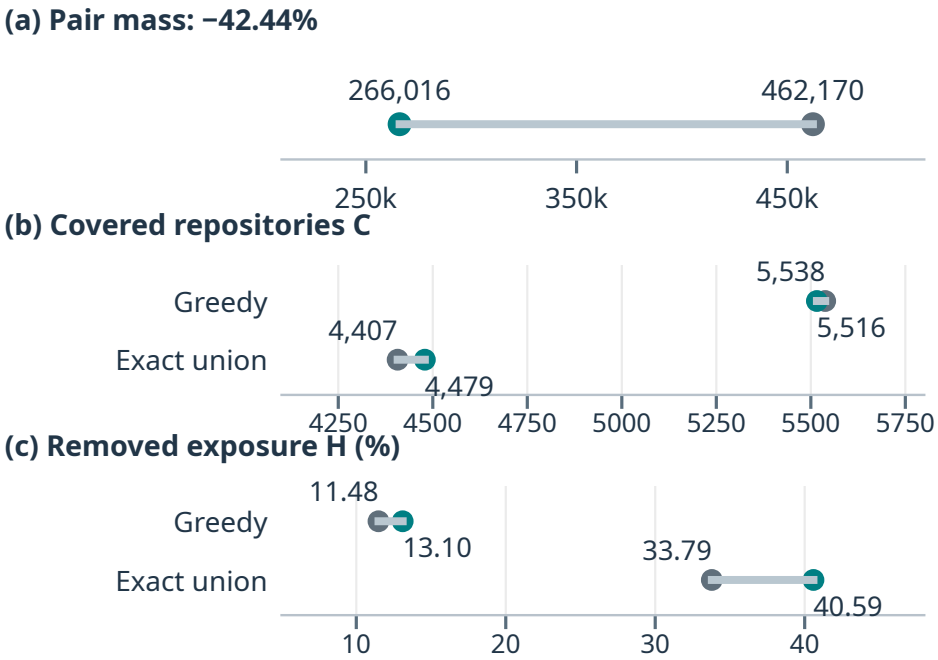


Fig. 9 Cross-representation divergence: preserving exact-version continuity removes some paths introduced by version merging and reduces total identity-repository exposure pairs by 42.44%, shifting maintainer leverage. Gray denotes name projection and teal denotes version continuity

name-projected rankings—an improvement of +13.43 **percentage points** (+64.2% relative gain).

5.3.2 Cross-Sectional Incident-Associated Illustrations

Table 4 details reach metrics and maintainer prioritization ranks across the 16 incident-associated records whose package names occur in the retained graph.

The individual-package comparison shows representation-dependent reach: `simple-swizzle` has 120 name-projected and 31 exact-version-continuous workflow endpoints; `chalk` has 1,125 and 587; `ansi-styles` has 1,376 and 740; `debug` has 1,462 and 1,162. These differences do not prove which paths an actual installation could execute.

The account-level example uses the same workflow endpoint definition. `qix` reaches 1,942 workflow repositories and ranks #7 by exact union; version continuity yields 1,476 and rank #5. It is beyond greedy's first 100 selections, while the hybrid places

Table 4: Observed workflow-repository reach and account rankings for 16 incident-associated records (15 package names). Reach and ranks use the retained cross-sectional graph, not pre-event installations

Package / record	Account (count)	Name reach	Version reach	Portfolio	Greedy	Union	Hybrid
`debug`	`qix` (2)	1,462	1,162	#2,298	#>100	#7	#35
`chalk`	`sindresorhus` (1)	1,125	587	#2	#1	#1	#1
`ansi-styles`	`sindresorhus` (1)	1,376	740	#2	#1	#1	#1
`simple-swizzle`	`qix` (1)	120	31	#2,298	#>100	#7	#35
`backslash`	`qix` (1)	0	0	#2,298	#>100	#7	#35
`eslint-config-prettier`	`lydell` (3)	10	10	#561	#41	#102	#31
`eslint-plugin-prettier`	`lydell` (6)	5	5	#561	#41	#102	#31
`@solana/web3.js`	`solana-devs` (1)	4	4	#181	#>100	#1,752	#465
`nx`	`jameshenry` (8)	2	2	#31	#>100	#835	#1,171
`node-ipc (v12)`	`riaevangelist` (12)	4	4	#2,562	#>100	#3,163	#1,274
`node-ipc (v10)`	`riaevangelist` (12)	4	4	#2,562	#>100	#3,163	#1,274
`@tanstack/history`	`tannerlinsley` (5)	1	1	#163	#>100	#1,423	#455
`@ctrl/tinycolor`	`scttcper` (1)	10	9	#5,765	#>100	#2,333	#1,227
`@opensearch-project/opensearch`	`peterzhuamazon` (3)	0	0	#2,548	#>100	#1,491	#2,323
`@mastra/core`	`tylerbarnes` (9)	0	0	#416	#>100	#6,314	#6,697
`echarts-for-react`	`atool` (1)	0	0	#398	#>100	#1,945	#747

Every account rank uses workflow-associated repository endpoints. The parenthesized value counts observed maintainers of that package. Membership is not evidence of historical publishing authority. Version labels and dependency counts are retained in the companion CSV. Package reach excludes the root; account portfolios include eligible seeds.

Table 5: All-source exclusion $P(S)$ optimization via greedy minimum-source heuristic versus baseline single-objective rankings

Budget	Direct $P(S)$ Heuristic			Greedy P	Union P	Gain vs Gr.	Gain vs Un.
$\lvert S\rvert$	P	C (%)	H (%)			ΔP	ΔP
10	**479**	55.4%	2.58%	351	343	+128	+136
25	**618**	59.1%	4.15%	514	427	+104	+191
50	**803**	64.2%	6.55%	638	441	+165	+362
100	**1,022**	70.0%	13.23%	800	534	+222	+488

it #35. This illustrates overlap in current retained memberships, not a successful pre-event forecast or historical publisher attribution.

Figure 10 separates the representation used for selection from the one used for evaluation. Comparisons of H within an evaluation row share a denominator; comparisons across rows change that denominator. A change in representation can therefore change both the selection and the quantity being measured.

6 Exclusion Heuristic Validation: RQ3

6.1 Baseline Gain and Component Ablations

On the original graph, the direct heuristic yields $P = 1{,}022$ at budget 100, compared with greedy coverage's 800 and exact union's 534. This gain comes with $C = 5{,}020$ and $H = 13.23\%$; it does not maximize every objective. Table 5 and Figure 11 report the multi-budget comparison.

The ablation analysis parameterizes the original lexicographic rule and reproduces its complete retained 100-account order before introducing variants. It changes the

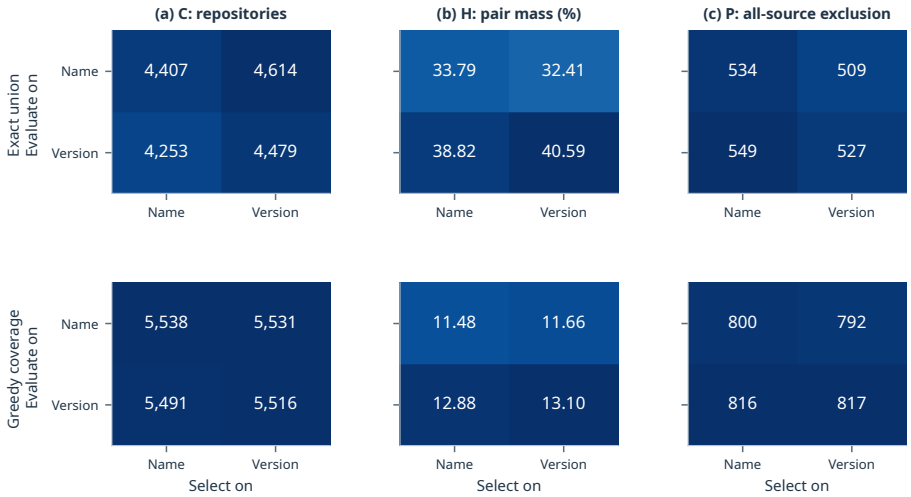


Fig. 10 Budget-100 selection/evaluation combinations for exact union and greedy. C and P count workflow repositories; H is divided by all account–repository pairs in the evaluation representation (462,170 for names; 266,016 for versions). Panels are separate metrics, not a common color scale

Table 6: Budget-100 component ablations on the original graph. Timing is one selection run in the same process

Rule	C	H (%)	P	Seconds
Original rule (cap 10)	5,020	13.23	1,022	19.56
Immediate + coverage	5,043	11.75	1,024	13.65
No coverage tie-break	5,020	13.23	1,022	13.64
Cap 1	5,020	13.23	1,022	4.96
Cap 5	5,020	13.23	1,022	11.45
Cap 20	5,020	13.23	1,022	17.63
No candidate cap	5,020	13.23	1,022	89.10

Immediate + coverage omits reciprocal remaining-source scoring and is identical to the no-reciprocal variant. Thirty seeded tie runs each give $P = 1{,}022$ at budget 100; this is selection variability on one graph, not ecosystem uncertainty.

candidate cap to 1, 5, 20 and unrestricted, removes reciprocal remaining-source scoring or the coverage tie-break, and repeats tied choices under 30 frozen seeds. These are component sensitivities, not independent ecosystem observations.

Removing reciprocal scoring gives $P = 1{,}024$, two more than the original rule, together with $C = 5{,}043$ and $H = 11.75\%$. The immediate-completion variant uses coverage then identity order to resolve zero-gain cases. Candidate-cap changes and removing the coverage tie-break retain the original budget-100 P on this graph. All 30 tie runs also yield $P = 1{,}022$. These outcomes provide no evidence that the reciprocal term improves original-graph exclusion; its usefulness must be assessed across other

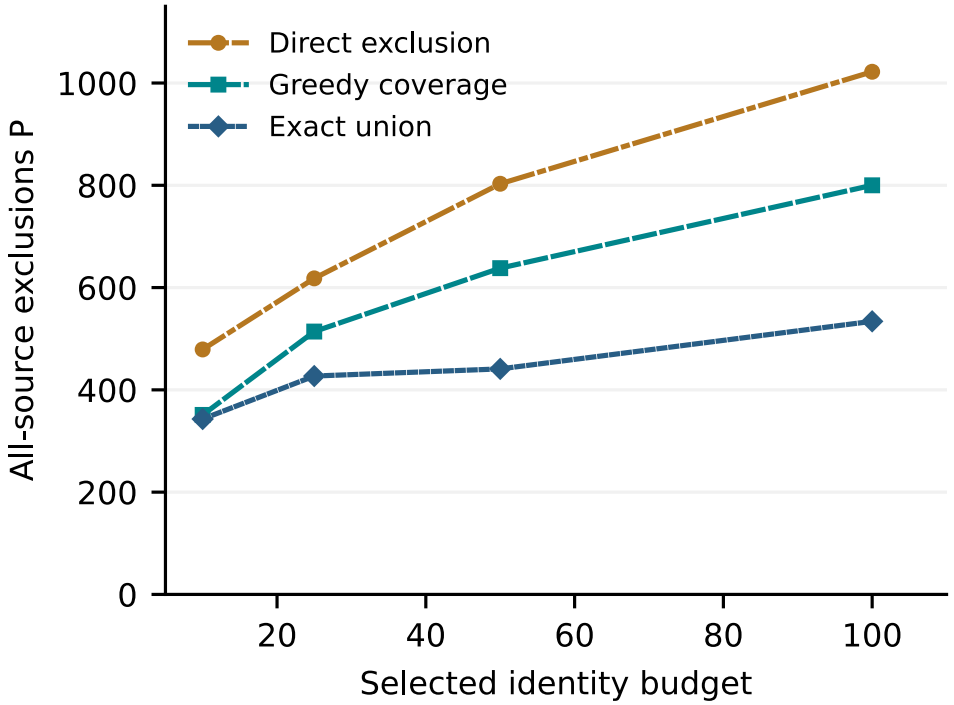


Fig. 11 All-source exclusion under the original heuristic and coverage/exposure baselines. Counts describe the retained graph under ideal selected-source hardening

observed populations. Single-run timing in Table 6 characterizes the implementation rather than a hardware-independent performance guarantee.

6.2 Candidate-Restricted Exact Benchmarks

We freeze 20 empirical candidate pools and 12 synthetic overlap instances, each with 20 eligible accounts and budgets 1, 3 and 5. Exhaustive enumeration evaluates every eligible subset. Empirical instances preserve complete endpoint source sets: sources outside the candidate pool remain unselected and can prevent exclusion. Candidate pools are constructed from low-cardinality source sets and reach-ranked fill accounts, so these are tractable diagnostic instances rather than a random sample of full-graph optimization problems. Table 7 reports the restricted-optimum comparisons.

The original heuristic reaches the restricted optimum in 86 of 96 settings. It matches 51 of 60 empirical settings and 35 of 36 synthetic settings. Its worst empirical deficit is 66 endpoints and its minimum achieved-to-optimal ratio is 16.46%; the worst synthetic deficit is one endpoint. Figure 3(b) shows the gap distributions. The 96 settings reuse 32 instances across budgets and are not independent replicates. These

Table 7: Exact exclusion benchmarks with 20 eligible accounts and budgets 1, 3 and 5

Instance family	Settings	Exact hits	Worst gap	Min ratio (%)
Empirical candidate pools	60	51	66	16.46
Synthetic overlap instances	36	35	1	93.33

Twenty empirical pools and twelve synthetic instances each supply three correlated budget settings. Outside-candidate sources remain unselected. Optima apply only to the restricted instance; no full-graph approximation guarantee follows.

diagnostics identify substantive failure cases; neither the hit count nor the synthetic results certify the original full graph's optimum or an approximation ratio.

6.3 Other Objective Choices

Standard scalarization exposes intermediate coverage–exposure choices, but a preferred weight requires review costs and stakeholder preferences that were not measured. Additional heterogeneous weights, strict thresholds and uniform-efficacy multipliers are reported in Appendix A. A threshold sensitivity is not a count of empirically protected repositories: source identities are not authenticated approval groups, and the study does not observe an enforced release policy.

7 New Bounded Acquisition: RQ4

7.1 Sampling, Timing and Observation Limits

Before new requests, we froze 100 roots sampled without replacement from the original graph's 22,483 ordinary-shaped identifiers. This frame excludes bundled-context strings but remains inherited from the original seeded expansion. It is not a random sample of all npm. Latest root versions are frozen on retrieval; deps.dev supplies version-specific dependency observations, and version-specific npm requests supply fresh metadata and maintainer associations. No old workflow flags or dependency rows are copied into the new graph.

The bounded acquisition retains at most 500 nodes per root and 2,500 exact (package, version) nodes globally, prioritizing sampled roots and deterministic traversal. It obtains dependency graphs for 94 of the 100 roots; the six failures remain recorded and are not interpreted as empty dependency trees. None of the acquired graphs loses nodes to these caps. One acquired root contains 15 node-level API error records, which are retained rather than interpreted as complete resolution. The retained union has 1,641 exact nodes, 1,476 identifiers and 4,357 version-edge rows. All 1,641 ordinary version-metadata requests succeed. The 1,051 observed usernames are associations, without a historical authority interpretation.

Of 95 roots with parsed GitHub mappings, 89 yield a commit observation and 74 yield a directory listing at that pinned commit. Requiring both dependency acquisition and directory observation retains 70 root records, which map to 66 repository endpoints; 65 have an observed source. The acquired directory is still a presence proxy. Missing mappings, commit observations, and directory observations remain

Table 8: Budget-100 results in the newly acquired bounded cohort: 66 workflow-associated repositories, 65 with observed sources

Representation	Rule	C	C (%)	H (%)	P
Name	Portfolio	41	62.12	13.51	6
Name	Exact union	42	63.64	44.95	4
Name	Greedy coverage	65	98.48	11.19	11
Name	Original direct P	62	93.94	17.72	34
Name	Immediate + coverage	63	95.45	14.23	28
Version	Portfolio	41	62.12	13.44	6
Version	Exact union	43	65.15	45.17	4
Version	Greedy coverage	65	98.48	11.28	11
Version	Original direct P	62	93.94	16.42	35
Version	Immediate + coverage	63	95.45	13.48	29

The cohort has 1,051 observed accounts. H denominators are 2,494 name pairs and 2,411 version pairs. Root selection comes from the original ordinary-name frame; acquisition failures remain recorded and six missing dependency graphs are not treated as empty graphs.

documented missingness rather than evidence of safety. Requests span a collection window rather than a shared instantaneous cutoff.

7.2 Within-Cohort Tradeoffs and Representation

Table 8 compares the budget-100 results within each representation.

At budget 100, greedy coverage reaches 65 of 66 endpoints, while exact union reaches 42. Their within-cohort H values are 11.19% and 44.95%. This budget selects 9.51% of the cohort's observed accounts, compared with 0.82% in the original graph; equal absolute budgets do not imply equal population shares. The original exclusion heuristic gives $P = 34$, compared with greedy's 11 and union's four; the immediate-completion variant gives 28. Thus the simpler rule's slight original-graph advantage does not transport to this cohort.

Figure 12 shows the within-cohort budget curves.

Version continuity reduces pair mass from 2,494 to 2,411, a 3.33% decrease. The original graph's 42.44% decrease therefore does not describe a universal representation correction. The two acquisitions differ in roots, observed graph extent, versions and dates; their difference cannot be attributed solely to temporal change. The repeated coverage–exposure ordering is descriptive evidence of transport under these explicit observation conditions, not a measured intervention effect.

7.3 Common Endpoints and Eligible Accounts

Table 9 reports results on the common population.

We also reoptimize on 65 common repository keys with 1,036 accounts eligible in both datasets. Each dataset retains its full observed source sets, including origins outside the common candidate pool. H retains the relevant dataset's pair denominator. At budget 100, coverage remains greater for greedy than exact union in both graphs; the direct heuristic gives the largest P among these three compared rules. Top-100 Jaccard overlaps between reoptimized selections are 0.667 for exact union, 0.835 for

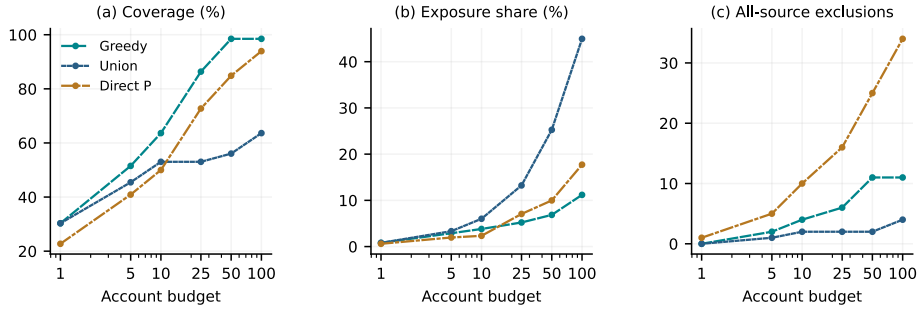


Fig. 12 New-cohort name-graph comparisons across the six frozen budgets. Coverage uses 66 observed repository endpoints, H uses 2,494 account–repository pairs, and P excludes the source-free endpoint. Curves reuse one acquired cohort and supply no npm-wide uncertainty estimate

Table 9: Reoptimized budget-100 selections on 65 common repository keys with 1,036 common selectable accounts

Dataset	Rule	C	H (%)	P	Pair mass
Original	Exact union	52	21.55	4	12,833
Original	Greedy coverage	65	6.09	8	12,833
Original	Original direct P	65	11.72	20	12,833
New cohort	Exact union	43	44.87	4	2,487
New cohort	Greedy coverage	64	11.22	11	2,487
New cohort	Original direct P	62	17.25	34	2,487

Complete source sets are preserved, including sources outside the common eligible account set. Pair denominators differ by observed graph, so H percentages are within-dataset quantities rather than a causal temporal effect.

greedy and 0.587 for direct exclusion. Shared observations and graph-bounding differences preclude treating these as independent ecosystem samples or a causal stability estimate.

8 Discussion

The practical choice begins with an objective. Coverage is suitable for distributing a review surface across distinct observed endpoints; additive H allocates selection toward account-origin pair mass under the stipulated weights; P requires selecting every observed source of a nonempty endpoint. The measured tradeoffs show that a defender cannot substitute one objective's score for another's. None of these quantities estimates how many incidents a real control prevents.

The additional experiments also distinguish useful evidence from mathematical expectation. Exact-union ranking's additive optimality follows from the objective; the empirical contribution is the associated coverage and exclusion cost. The new cohort repeats that direction while changing effect sizes. A representation effect of 42.44% in

one union and 3.33% in the other warrants reporting population and version context whenever reach is compared.

The exclusion heuristic illustrates why added complexity needs empirical justification. Reciprocal scoring is unnecessary for its original-graph P improvement, yet helps relative to the immediate variant in the new cohort. Candidate-restricted exact instances expose large empirical gaps despite frequent optimum matches. The present results support a transparent heuristic comparison, without a universal recommended cap or algorithmic guarantee.

The endpoint audit provides reproducible, commit-pinned evidence collection. Its automated indicators were not manually validated and do not establish mapping accuracy, workflow execution or historical validity. A deployment recommendation would additionally require observed authorization paths, intervention costs, feasible control policies and outcome evaluation. These missing measurements are separate from whether the retained graph computations are correct.

9 Threats to Validity and Research Ethics

We organize limitations around construct, internal, external, and conclusion validity, informed by empirical reporting guidance (Ralph et al. 2020).

9.1 Construct Validity

Registry maintainer arrays supply username associations, not historical publication authority, token scope, account independence, or a peer-review policy. One maintainer does not prove absence of human review; several do not prove enforced review. Uniform and heterogeneous source weights are modeling assumptions. The perfect hardening model removes selected account origins, not package nodes or paths from other origins.

The workflow observation is a directory lookup at HEAD. The original acquisition did not retain workflow YAML, job execution, permission scopes, historical release links, or consumer lockfiles. Dependency and workflow association therefore establishes neither installation nor vulnerable-code execution or secret access. CI dependency studies (Weeraddana et al. 2024) and finer-grained vulnerability analysis (Shen et al. 2025) motivate those distinctions without supplying execution evidence for our endpoints. The new endpoint audit separately retains current commit-pinned workflows and selected manifests/lockfiles. It does not recover the missing original evidence. The automated indicators were not manually validated. Regular-expression and YAML-based extraction may miss reusable actions, infer the wrong working directory, or flag shell text that does not execute an install. These errors can affect the evidence-subset diagnostics.

The main exact-version graph is an observed deps.dev edge union, while the temporal companion is a bounded highest-satisfying SemVer approximation. They are different representations. Neither reconstructs npm hoisting, installed versions, overrides, or complete historical dependency trees. Runtime vulnerabilities such as the Node.js Brotli disclosure (The Node.js Project 2026) are outside the package graph. Residual thresholds are hypothetical source-count rules, without signer groups, role constraints, or measured enforcement.

9.2 Internal and Observational Validity

Independent traversals, transposed source incidence, small-graph cases, and generated result checks validate computations on retained evidence. They cannot recover omitted observations. SCC condensation preserves reach; it is not a source of measured cycle-removal benefit. Ties use ascending username order, not portfolio size, except where portfolio size defines the selection rule itself.

Repository normalization is lexical. Excluding inconsistent positive observations tests sensitivity; it does not validate the other remote mappings or repair historical flags. Both main representations mix contexts and acquisition dates. Unknown workflow states and ambiguous zeros can omit eligible endpoints. Historical availability counts describe retained records, including a repeated tuple, and do not establish independent incidents or verified release authority.

9.3 External and Conclusion Validity

The high-impact-seeded expansion is neither registry-wide nor a random sample of npm installations. Public workflow-associated repositories cannot represent private consumer infrastructure without further evidence. Results describe this graph and explicit scenarios. They supply no measured compromise probabilities or causal control effects. The new 100-root acquisition inherits the original ordinary-name sampling frame and observes a bounded cohort with only 66 eligible repository endpoints. Six failed dependency acquisitions and missing GitHub observations limit transport. Comparing full and common populations exposes scope differences but does not isolate time from population or graph extent. Two acquisitions do not supply independent temporal replicates. The small exact benchmarks deliberately use tractable candidate pools and retain outside sources; they cannot establish global approximation performance.

Random orders quantify selection variability. Orders are reused across nested budgets and endpoint definitions, so the resulting 6,000 rows are not independent ecosystem samples. Monte Carlo errors describe simulation means, not npm-wide uncertainty. The seven scalarization settings are exploratory tradeoff points, without a certified global Pareto frontier or an empirically estimated stakeholder preference. Heterogeneous weights are structural sensitivities, not fitted risk priors. Claims about efficacy require a separate intervention study.

9.4 Ethics and Reporting

The analyses read retained public metadata and execute no package payloads, credential probes, or exploitation. Prospective captures omit email addresses and package bodies. Public usernames illustrate structural overlap without implying present-day insecurity, negligence, or historical publishing authority. Incident associations do not validate prediction. Artifact checksums identify retained bytes rather than certify authenticity.

10 Conclusion

Maintainer selection depends on the quantity a review program seeks to improve. In the original graph, budget-100 greedy covers 5,538 repositories and exact union 4,407, with uniform-scenario exposure reductions of 11.48% and 33.79%. The direct exclusion heuristic increases P from greedy's 800 to 1,022, but omitting reciprocal scoring yields 1,024. Candidate-cap and tie sensitivities leave the original heuristic's final P unchanged, while restricted exact instances reveal deficits as large as 66 endpoints.

A fresh 100-root acquisition supplies 66 workflow-associated repository endpoints and repeats the coverage–exposure tradeoff. Greedy reaches 65 and exact union 42, while the direct heuristic excludes all sources for 34 endpoints. Version continuity decreases pair mass by 3.33% in this cohort, compared with 42.44% in the original union. The shared direction and differing magnitude support explicit population, representation and denominator choices rather than a universal numeric correction or preferred account list.

The automated 120-endpoint evidence audit acquires 115 pinned commits. Its indicators were not manually validated and establish neither executed dependencies nor historical mapping precision. The appendices report historical metadata reconstruction and metadata-acquisition feasibility; no forecasting model is evaluated. The completed computations support structural prioritization comparisons and documented failure cases; evaluating actual control effectiveness requires additional authority, installation and intervention evidence.

Statements and Declarations

Funding

This research received no external funding.

Competing Interests

Both authors declare that they have no financial or non-financial competing interests relevant to this work.

Data Availability

The retained input archive, new observation records, derived analysis tables, protocols and checksums (Hasan and Islam 2026) are available in the versioned research prerelease `research-validation-2026-10-28`: https://github.com/hasanshahriar32/npm-supply-chain-research/releases/tag/research-validation-2026-10-28. The companion archive and its checksum-verified original input are linked at a fixed repository commit. Third-party workflow, manifest and lockfile bodies are referenced by pinned source URLs and hashes; the release contains derived audit observations.

Code Availability

The same versioned artifact provides collection and analysis scripts, exact benchmark inputs, numerical consistency checks and figure-generation code. Its README

describes checksum verification and offline numerical reproduction after downloading the original input. The numerical path uses captured observations and does not repeat live API requests. Repeating acquisition may return different metadata. Third-party source bodies can be inspected through the pinned URLs; offline numerical reproduction uses the released derived observations.

Ethics and Consent

The study analyzes public software-ecosystem metadata and does not involve recruited human participants, interventions on individuals, or execution of package payloads. No participant consent was collected. Usernames are used as observed graph identifiers, without inferring misconduct or present-day account security.

References

Bloom Security (2026) SlugFest: Millions of AI agents install npm packages exposed to malicious takeover through GitHub. URL https://bloom.security/blog/slugfest-millions-of-ai-agents-download-exposed-npm-packages, researcher disclosure; accessed October 25, 2026

Cohen E (2026) Open and shut: Remote code execution in opencode via MCP OAuth. Pillar Security research disclosure, URL https://www.pillar.security/blog/open-and-shut-remote-code-execution-in-opencode-via-mcp-oauth, october 19 disclosure; September 28 fix; accessed October 25, 2026

GitHub (2026) Unvalidated npm trusted publishing configurations now expire. URL https://github.blog/changelog/2026-10-02-unvalidated-npm-trusted-publishing-configurations-now-expire/, october 2 changelog; accessed October 25, 2026

Guo W, Chen Z, Xu Z, et al (2026) How effective are NPM malicious package detectors? a large-scale empirical study. arXiv:2603.27549v2, URL https://arxiv.org/abs/2603.27549v2, revised August 5, 2026

Hasan S, Islam MA (2026) npm supply-chain research: Validation data and code. GitHub research artifact, research-validation-2026-10-28, URL https://github.com/hasanshahriar32/npm-supply-chain-research/releases/tag/research-validation-2026-10-28, versioned research artifact; accessed October 28, 2026

He H, Vasilescu B, Kästner C (2025) Pinning is futile: You need more than local dependency versioning to defend against supply chain attacks. Proceedings of the ACM on Software Engineering 2(FSE):266–289. https://doi.org/10.1145/3715728, URL https://arxiv.org/abs/2502.06662

Jayasuriya D, Ou S, Hegde S, et al (2025) An extended study of syntactic breaking changes in the wild. Empirical Software Engineering 30(2):42. https://doi.org/10.1007/s10664-024-10563-4

Jensen O, Wang M (2026) The attack chain behind the v1.107.0 LiteLLM release. URL https://docs.litellm.ai/blog/security-incident-oct-21-2026-attack-chain, october 23 final investigation; accessed October 25, 2026

Kalu KG, Singla T, Okafor C, et al (2025) An industry interview study of software signing for supply chain security. In: 34th USENIX Security Symposium (USENIX Security 25). USENIX Association, pp 81–100, URL https://www.usenix.org/conference/usenixsecurity25/presentation/kalu

Koishybayev I, Nahapetyan A, Zachariah R, et al (2022) Characterizing the security of GitHub CI workflows. In: 31st USENIX Security Symposium. USENIX Association, pp 2747–2763, URL https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev

LiteLLM Project (2026) Contained: Oct 21, 2026 LiteLLM release v1.107.0 compromised. URL https://docs.litellm.ai/blog/security-incident-oct-21-2026, artifact scope and availability window; accessed October 25, 2026

Liu C, Chen S, Fan L, et al (2022) Demystifying the vulnerability propagation and its evolution via dependency trees in the NPM ecosystem. In: 44th International Conference on Software Engineering (ICSE), https://doi.org/10.1145/3510003.3510142, URL https://arxiv.org/abs/2201.03981v2

Muralee S, Koishybayev I, Nahapetyan A, et al (2023) ARGUS: A framework for staged static taint analysis of GitHub workflows and actions. In: 32nd USENIX Security Symposium. USENIX Association, pp 6983–7000, URL https://www.usenix.org/conference/usenixsecurity23/presentation/muralee

Nemhauser GL, Wolsey LA, Fisher ML (1978) An analysis of approximations for maximizing submodular set functions—I. Mathematical Programming 14:265–294. https://doi.org/10.1007/BF01588971

Ohm M, Plate H, Sykosch A, et al (2020) Backstabber's knife collection: A review of open source software supply chain attacks. arXiv:2005.09535, URL https://arxiv.org/abs/2005.09535, DIMVA 2020 preprint

Open Source Insights (2026) deps.dev API v3: GetDependencies. Official API documentation, URL https://docs.deps.dev/api/v3/, accessed October 20, 2026

Pinckney D, Cassano F, Guha A, et al (2023) npm-follower: A complete dataset tracking the NPM ecosystem. arXiv:2308.12545, URL https://arxiv.org/abs/2308.12545

Rahman I, Marley J, Paramitha R, et al (2026) CHRONO-RESOLUTION: A dependency resolution dataset at release points for npm, PyPI, and crates.io packages. arXiv:2607.15315, URL https://arxiv.org/abs/2607.15315, ICSME 2026 Tool Demonstration and Data Showcase Track

Ralph P, et al (2020) Empirical standards for software engineering research. arXiv:2010.03525, URL https://arxiv.org/abs/2010.03525

Schorlemmer TR, Kalu KG, Chigges L, et al (2024) Signing in four public software package registries: Quantity, quality, and influencing factors. In: IEEE Symposium on Security and Privacy (S&P), pp 1160–1178, https://doi.org/10.1109/SP54263.2024.00215, URL https://arxiv.org/abs/2401.14635

Sejfia A, Schäfer M (2022) Practical automated detection of malicious npm packages. In: Proceedings of the 44th International Conference on Software Engineering, pp 1681–1692, https://doi.org/10.1145/3510003.3510104

Shafin AA, Ahmed KM (2026) PatchRisk: Forecasting future vulnerability exposure in open-source dependency networks. arXiv:2609.13719, URL https://arxiv.org/abs/2609.13719, WI-IAT 2026 preprint

Shen Y, Gao X, Sun H, et al (2025) Understanding vulnerabilities in software supply chains. Empirical Software Engineering 30(1):20. https://doi.org/10.1007/s10664-024-10581-2, URL https://gaoxiang9430.github.io/papers/emse.pdf

The Node.js Project (2026) Wednesday, october 21, 2026 security releases. URL https://nodejs.org/en/blog/vulnerability/october-2026-security-releases, cVE-2026-111497 rated High; accessed October 25, 2026

Wang Z (2026) Defending the core: A centrality-based protection strategy for supply chain security in npm dependency network. arXiv:2606.14036, URL https://arxiv.org/abs/2606.14036, preprint

Weeraddana NR, Alfadel M, McIntosh S (2024) Dependency-induced waste in continuous integration: An empirical study of unused dependencies in the npm ecosystem. Proceedings of the ACM on Software Engineering 1(FSE):2632–2655. https://doi.org/10.1145/3660823

Zahan N, Zimmermann T, Godefroid P, et al (2022) What are weak links in the npm supply chain? In: Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice, pp 331–340, https://doi.org/10.1145/3510457.3513044

Zimmermann M, Staicu CA, Tenny C, et al (2019) Small world with high risks: A study of security threats in the npm ecosystem. In: 28th USENIX Security Symposium. USENIX Association, pp 995–1010, URL https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman

Table 10: Observed greedy tradeoffs across scalarization weight α at budget $|S| = 100$ ($|R_{\text{wf}}| = 7{,}171$)

	Package-Projected Graph				Version-Continuous Graph			
Scalarization α	C	C (%)	H (%)	P	C	C (%)	H (%)	P
0.00 (Exact Union)	4,407	61.46%	33.79%	534	4,479	62.46%	40.59%	527
0.10	4,591	64.02%	33.65%	530	4,714	65.74%	40.37%	553
0.25	4,809	67.06%	32.98%	604	4,855	67.70%	39.94%	634
0.50 (Equal weighting)	5,020	70.00%	31.19%	626	5,000	69.73%	38.65%	648
0.75	5,403	75.35%	21.69%	779	5,239	73.06%	31.75%	695
0.90	5,529	77.10%	14.15%	794	5,509	76.82%	14.97%	826
1.00 (Greedy Coverage)	5,538	77.23%	11.48%	800	5,516	76.92%	13.10%	817

A Extended Objective Sensitivities

A.1 Exploratory Scalarization

We evaluate $J_\alpha(S) = \alpha C(S)/|R_{\text{wf}}| + (1-\alpha)H(S)$ at seven weights, $\alpha \in \{0, 0.10, 0.25, 0.50, 0.75, 0.90, 1\}$. These greedy solutions trace an observed tradeoff curve. Neither the finite weight sweep nor the approximation guarantee establishes the global Pareto frontier or a universally preferred weight.

Table 10 and Figure 13 summarize the evaluated scalarization settings.

At $\alpha = 0.50$, the name graph gives $C = 5{,}020$, $H = 31.19\%$, and $P = 626$; the version graph gives $C = 5{,}000$, $H = 38.65\%$, and $P = 648$. The name result retains 90.65% of greedy's baseline coverage and 92.31% of exact union's H. Relative to greedy, it loses 518 covered repositories and gains 19.71 percentage points of H. Relative to exact union, it gains 613 repositories and loses 2.60 points of H. A preferred operating point requires review costs and stakeholder preferences that were not measured here.

A.2 Strict Zero-Anchored Threshold Sensitivity

Multi-token incidents motivate asking how many unhardened source identities remain, but token count is not a measured release quorum. In LiteLLM's final account, credentials served different roles, including deletion of records after publication (Jensen and Wang 2026). We evaluate the explicit strict rule in Eq. 6, retaining only the 7,158 endpoints with observed sources and excluding all 13 source-free endpoints:

$$P_k(S) = \left|\{r \in R_+ : t_r(S) = 0 \ \vee \ (n_r \geq k \ \wedge \ t_r(S) < k)\}\right|. \tag{12}$$

The unguarded condition $t_r(S) < k$ credits every endpoint with $n_r < k$ even at zero hardening. In the strict model, a single-source endpoint must have its sole maintainer hardened at every threshold, and any endpoint with $n_r < k$ requires complete exclusion. This ensures a zero baseline for each tested k; it is not equivalent to subtracting the old zero-budget count.

A protection interpretation still requires a measured policy in which every successful attack needs the designated number of distinct upstream identities with no bypass. Our graph lacks package-level signer groups, role constraints, and bypass privileges.

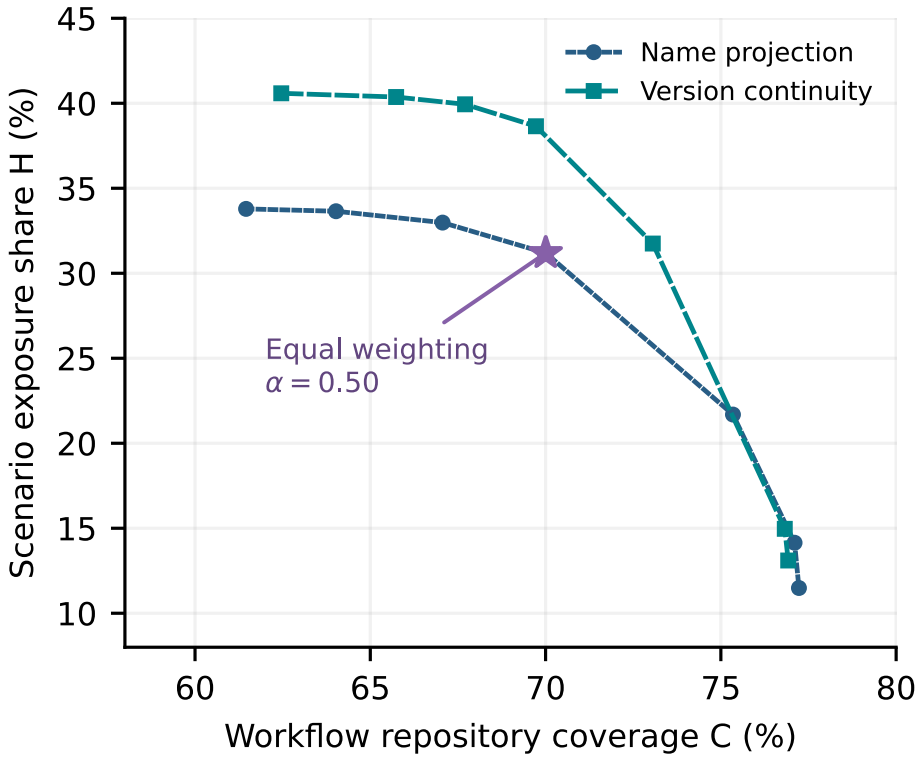


Fig. 13 Observed greedy scalarization results at budget 100. Connecting seven evaluated settings guides the eye; the curve is not a certified global Pareto frontier. Equal weighting is a sensitivity choice

Two unrelated single-maintainer dependencies can each poison a consumer even if their combined source cardinality is two. Table 11 is therefore a structural sensitivity analysis, not a count of empirically protected repositories.

At budget 100, Pareto Hybrid has $P_1 = 626$, $P_2 = 961$, $P_3 = 1{,}018$, $P_5 = 1{,}063$. Greedy Coverage has $P_1 = 800$, $P_2 = 1{,}145$, $P_3 = 1{,}125$, $P_5 = 1{,}105$. Exact Union has $P_1 = 534$, $P_2 = 793$, $P_3 = 901$, $P_5 = 1{,}022$. Direct P has $P_1 = 1{,}022$, $P_2 = 1{,}169$, $P_3 = 1{,}286$, $P_5 = 1{,}383$. The source-cardinality distribution is $n_r = 0$: 13; $n_r = 1$: 1,716; $n_r = 2$: 546; $n_r = 3$: 354; $n_r = 4$: 347; and $n_r \geq 5$: 4,195. All strict zero-budget counts are zero.

The same ordered maintainer selections are reused at all thresholds, and the Direct P strategy continues to optimize its original $k = 1$ heuristic. Counts are nondecreasing with nested hardening budgets for a fixed k. Comparisons across k represent different hypothetical policies and need not be monotone, because endpoints with $n_r < k$ revert to complete-exclusion requirements.

Table 11: Strict zero-anchored threshold counts $P_k(S)$ and coverage across maintainer hardening strategies ($|R_{\text{wf}}| = 7{,}171$)

Prioritization Strategy	Budget $\lvert S\rvert$	$C(S)$	C (%)	$H(S)$ (%)	$P_1(S)$ $(k=1)$	$P_2(S)$ $(k=2)$	$P_3(S)$ $(k=3)$	$P_5(S)$ $(k=5)$
Portfolio Size	10	3,435	47.90%	1.59%	391	512	488	521
	25	4,067	56.71%	4.38%	430	602	657	661
	50	4,446	62.00%	6.14%	503	718	777	784
	100	4,579	63.85%	7.96%	577	774	858	892
Greedy Coverage	10	4,331	60.40%	3.79%	351	477	495	468
	25	4,854	67.69%	6.08%	514	756	732	727
	50	5,196	72.46%	8.73%	638	936	921	907
	100	5,538	77.23%	11.48%	800	1,145	1,125	1,105
Exact Union	10	3,910	54.53%	4.48%	343	440	466	485
	25	4,284	59.74%	9.86%	427	637	719	694
	50	4,321	60.26%	18.09%	441	655	734	787
	100	4,407	61.46%	33.79%	534	793	901	1,022
Pareto Hybrid ($\alpha = 0.50$)	10	4,331	60.40%	3.79%	351	477	495	468
	25	4,795	66.87%	7.68%	498	706	734	729
	50	5,001	69.74%	14.96%	605	907	985	1,019
	100	5,020	70.00%	31.19%	626	961	1,018	1,063
Direct Supermodular $P(S)$	10	3,976	55.45%	2.58%	479	614	609	586
	25	4,239	59.11%	4.15%	618	743	774	779
	50	4,604	64.20%	6.55%	803	948	998	1,016
	100	5,020	70.00%	13.23%	1,022	1,169	1,286	1,383

Table 12: Budget-100 outcomes under assumed weights. Each H column has its own stipulated denominator; weights are not estimated compromise probabilities

Selection	C	H (%)	Account-sole (%)	Log (%)	Package-sole (%)	P
Exact union	4,407	33.79	77.30	40.15	27.32	534
Greedy coverage	5,538	11.48	74.19	18.52	23.40	800
Portfolio	4,579	7.96	70.26	15.06	13.72	577
Hybrid (0.50)	5,020	31.19	80.41	38.93	28.51	626
Account-sole aware	4,919	22.59	88.04	33.09	44.53	707
Log-portfolio aware	4,603	31.91	81.42	42.91	33.17	514
Package-sole aware	4,718	19.39	80.03	23.61	67.37	642

For an additional hypothetical sensitivity, uniform control efficacy η gives $H_\eta(S) = \eta H(S)$. At $\eta = 0.90$, Pareto Hybrid yields 28.07% effective scenario exposure reduction. This uniform multiplier preserves all H rankings and does not estimate a real bypass rate, correlate account failures, or model CI compromise. Its values are assumptions, not incident-derived estimates.

A.3 Heterogeneous Weight Sensitivities

Table 12 compares the assumed weightings.

The account-sole weight increases an account's contribution according to its number of sole-maintained packages; it does not restrict its reachable endpoints to those

Table 13: Retained bounded registry-SemVer reconstruction manifest. The recorded resolution share is conditional on attempted expansions within the bounds

Recorded dimension	Value
Retained root records	45
Distinct event/root/cutoff tuples	44
Recorded edge attempts	1,848
Resolved recorded edges	1,841
Node counts summed over roots	1,272
Unresolved expansions	7
Resolved share of recorded attempts	99.62%
Maximum expansion depth	3
Node cap per root	250
Roots reaching cap	1

Only regular dependencies are included; depth is capped at three and nodes at 250 per root. Highest retained cutoff-valid satisfying versions are used. These are bounded approximations, not lockfile reconstructions or completeness guarantees. Raw-response checksums establish retained-object identity, not publisher authenticity.

packages. Ranking by that weighted reach captures 88.04% of the stipulated account-sole mass. This is concentration under an assumed weighting, not nine-tenths of measured ecosystem risk.

The separate package-sole quantity unions reach only from each account's sole-maintained packages. Under that definition, unweighted greedy captures 23.40%, unweighted exact union 27.32%, and package-sole-aware ranking 67.37%. The latter gain is 40.05 percentage points relative to unweighted exact union. Its uniform coverage is 4,718 repositories and $P = 642$. These results distinguish account weighting from package-restricted reach. A single observed maintainer is not evidence about review quality or an empirically fitted probability of compromise.

B Historical Metadata and Acquisition Feasibility

The historical companion asks what retained registry metadata can support before an event cutoff. It is separate from the cross-sectional reverse-reach experiment. It does not test whether the primary rankings would have predicted historical compromise.

B.1 Bounded Cutoff-SemVer Reconstruction

For each retained root record, the reconstruction chooses the highest currently retained, non-prerelease version satisfying a runtime dependency range and published by the root cutoff. Expansion stops at depth three or 250 nodes per root. Peer, optional, development, bundled, workspace, override, git, and tarball dependency behavior is excluded. Deleted historical versions may be unavailable; present-day retention cannot recreate every past registry state.

Table 13 summarizes the bounded reconstruction.

The source contains 45 root records but 44 distinct event/root/cutoff tuples: one `node-ipc` tuple is repeated. We report retained-record counts without treating them as independent incidents. Of 1,848 recorded edge attempts, 1,841 resolve and seven have unavailable target registries, yielding 99.62% among recorded attempts. This percentage excludes unvisited expansions beyond the bounds. The 1,272 node count

Table 14: Field availability in 45 retained pre-event version-object records. Comparison availability is not continuity, equality, or authenticated release authority

Metadata availability	Count	Share
Retained selected-version objects	45	100.0%
Immediate predecessor available	44	97.8%
Repository-field comparison available	42	93.3%
gitHead-field comparison available	34	75.6%
publishConfig comparison available	11	24.4%
Selected dist.integrity present	45	100.0%
Selected embedded maintainers present	45	100.0%

The 45 records include a repeated event/root/cutoff tuple; they are not independent incidents. Embedded maintainer metadata is not a signature. No version-specific provenance endpoint requests, bundle retrievals, signature verifications, or attestation-state classifications were performed. Historical attestation presence/absence is unknown, not 0/45 absent.

is summed across roots, not globally deduplicated. One record reaches the node cap. These measurements do not establish full-tree completeness, downstream population reconstruction, or unbiased historical rankings.

B.2 Release-Field Availability and Authority Limits

Table 14 reports the availability of release fields.

The availability audit finds predecessor metadata for 44 of 45 retained records, repository-field comparison availability for 42, and `gitHead`-field comparison availability for 34. Availability does not show that URLs stayed equal, commits were valid or available, or a publisher retained authority. The audit classified no field deltas, built no authority scores, and evaluated no release-gating control.

All 45 records contain `dist.integrity` and embedded maintainer metadata. A distribution hash identifies expected artifact bytes; an embedded maintainer array is not a cryptographic signature. Neither proves a benign or reproducible build. The attestation access assessment made no version-specific requests, retrieved no bundles, ran no signature verification, and classified no attestation states. Historical attestation presence is therefore *unknown*; zero retrievals cannot be reported as zero attestations. Signing studies motivate the need for stronger evidence (Kalu et al. 2025; Schorlemmer et al. 2024), without supplying the missing measurements for this cohort.

The LiteLLM source-tag/artifact discrepancy (LiteLLM Project 2026) illustrates why artifact identity, intended source, and release authority must be evaluated separately. Contextual reports do not add observations to the 45-record companion.

B.3 Metadata-Acquisition Pilot

A separate metadata pilot sampled 100 identifiers uniformly without replacement from the original 25,228-identifier frame (seed 20261020). Of 92 ordinary-shaped names, 90 yielded metadata and two returned HTTP 404; all eight bundled-context identifiers returned HTTP 405. No identifier was replaced. The acquisition cutoff was October 20, 2026, 00:48:38 Asia/Dhaka. The pilot characterizes retrieval feasibility and sampling-frame defects; it includes no measured incident outcomes or forecasting evaluation.